POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Fluid-Structure Interaction

Master's degree project in
Numerical Methods for Partial Differential Equations

**Nicola Chiarani 11001700, Emanuele Dajko 11162942,
Claudio Tessa 11131286, Davide Trombetta 10790028**

Advisor:
Prof. Alfio Maria Quarteroni

Co-advisors:
Prof. Michele Bucelli

Academic year:
2025-2026

Abstract: This report documents the implementation of a two-dimensional steady fluid-structure interaction solver developed for Project 9 of the Numerical Methods for Partial Differential Equations course. The assigned model couples an incompressible Stokes flow with a linearly elastic solid on a simplicial mesh generated by Gmsh and imported in deal.II. The implementation follows the small-displacement setting of the brief: the fluid is solved on a fixed domain, the velocity is constrained to vanish on the fluid-solid interface, and the fluid traction loads the solid. At the discrete level, the unknowns are assembled in a global block system by combining Taylor-Hood elements for the fluid with piecewise quadratic finite elements for the solid through an `hp::FECollection` and `FE_Nothing` components. The solver uses MPI-parallel deal.II/Trilinos data structures, a distributed block sparse matrix, a Schur-type preconditioned FGMRES solve for the Stokes block, and a CG–AMG solve for the solid block. Numerical tests on 1, 2, 3, and 4 MPI ranks show rank-independent results up to solver tolerance: the relative L^2 discrepancy between the 1-rank and 2-rank runs is approximately 7.39×10^{-6}. The implementation is therefore validated with respect to distributed consistency, and its parallel performance is assessed through a dedicated strong-scalability study.

Key-words: fluid-structure interaction, Stokes equations, linear elasticity, finite elements, deal.II, Gmsh

1. Introduction

Fluid-structure interaction (FSI) problems describe the interaction between a flowing fluid and a deformable solid. They arise in many engineering applications, including aeroelasticity, biomedical flows, valves, membranes, and compliant structures. In the most general setting, the fluid stresses deform the solid and the resulting displacement modifies the fluid domain, leading to a nonlinear moving-interface problem.
Project 9 asks instead for a steady two-dimensional linear FSI solver under a small-displacement assumption: the fluid domain remains fixed, the fluid velocity is approximated by zero on the fluid-solid interface, and the solid is loaded by the fluid traction. In this model the fluid equations do not depend on the solid displacement, so the monolithic algebraic system is expected to be block lower triangular. The present implementation follows that structure directly, rather than approximating a more general ALE or moving-domain model.
The objective of the work is fourfold: first, to build a complete numerical workflow in `C++` using the finite element library `deal.II` [1]; second, to implement the solver with MPI-parallel Trilinos linear algebra; third, to verify

that the distributed code produces rank-independent numerical results together with meaningful post-processed fields; fourth, to analyze the strong scaling and speedup of the implemented system.

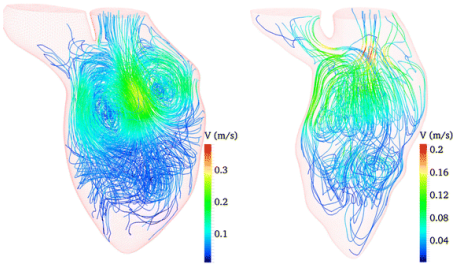


Figure 1: An example application area for fluid-structure interaction: blood flow in deformable biological structures.

1.1. Report organization

The report is organized as follows. Section 2 summarizes the `GitHub` repository layout. Section 3 describes the geometry, the mesh labels, and the mathematical model implemented in the code. Section 4 presents the discrete formulation and the block structure of the algebraic system. Section 5 discusses the main implementation choices in `deal.II` and Trilinos. Section 6 reports the numerical validation, with particular attention to MPI-rank consistency. Section 7 analyzes the strong-scaling behavior of the implementation.

2. Repository Structure

The repository is organized to separate the solver, the mesh-generation workflow, and the documentation:

```
.
|-- .github/workflows/
|-- Paper/
|-- common/
|-- mesh/
|-- scripts/
|-- src/
|-- .gitignore
|-- CMakeLists.txt
|-- CONTRIBUTION.md
|-- README.md
```

The role of the main folders is the following:

- `src/`: contains the implementation of the numerical solver. The core class is declared in `FSIProblem.hpp` and implemented in `FSIProblem.cpp`; the executable entry point is `exerciseFSI.cpp`.
- `mesh/`: contains the geometry file `mesh_fsi_geo.geo` and a `Makefile` used to generate the simplicial mesh `mesh_fsi.msh` with Gmsh in MSH v2 format.
- `Paper/`: contains the presented report.

3. Model Problem and Mesh

3.1. Geometry and Subdomains

The computational geometry is shown in Figure 2. It is a two-dimensional domain composed of two regions: a fluid subdomain and a solid subdomain. The geometry is created in Gmsh [3] and then imported into `deal.II` through `GridIn::read_msh`.

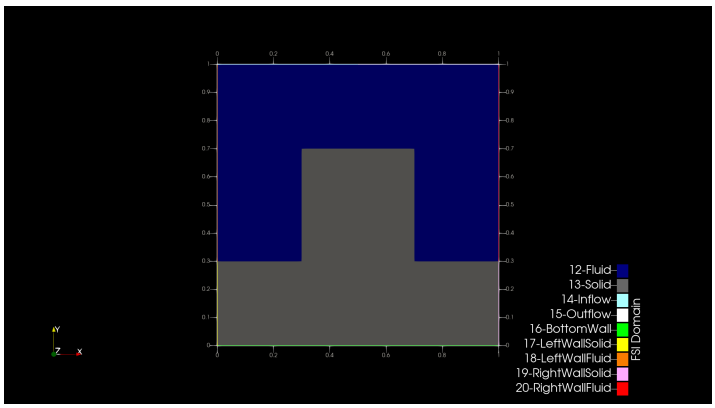


Figure 2: Computational domain used by the solver. The geometry is partitioned into a fluid region (blue) and a solid region (grey).

The code identifies the two materials by integer tags:

- Fluid = 12,
- Solid = 13.

The internal interface is not assigned a dedicated boundary id in the mesh file; instead, it is detected during setup by checking neighboring cells with different active finite elements.

3.2. Boundary labels and boundary conditions

The boundary ids used in the implementation are summarized in Table 1.

Table 1: Boundary identifiers and imposed conditions in the implementation.

ID	Boundary name	Condition in code
14	`Inflow`	Prescribed parabolic inflow velocity
15	`Outflow`	Natural Stokes outflow
16	`BottomWall`	Zero solid displacement
17	`LeftWallSolid`	Natural solid boundary
18	`LeftWallFluid`	Natural fluid velocity
19	`RightWallSolid`	Natural solid boundary
20	`RightWallFluid`	Natural fluid velocity

On the fluid-solid interface Σ, the solver imposes a homogeneous velocity constraint on the fluid side,

$$\boldsymbol{u} = \mathbf{0} \qquad \text{on } \Sigma,$$

while the fluid traction is transferred to the solid through the interface term assembled in the solid equations. Since the project assumes small displacements, the fluid domain is not updated by the solid deformation; the resulting lower-triangular coupling is therefore consistent with the assigned model rather than a defect of the implementation.

3.3. Mesh generation

The mesh is generated from `mesh/mesh_fsi_geo.geo` with Gmsh. In order to be readable by `deal.II` in the adopted workflow, the mesh must be written in MSH v2 format. The repository Makefile uses the command

```
gmsh -2 mesh_fsi_geo.geo -format msh2 -nopopup -o mesh_fsi.msh
```

which produces a valid triangular mesh in the `mesh/` directory. The executable expects to find the mesh at relative path `../mesh/mesh_fsi.msh` when launched from the `build/` directory.

4. Continuous and Discrete Formulation

4.1. Continuous model

The project statement prescribes the steady FSI model

$$\begin{cases} -\nu\Delta \boldsymbol{u} + \nabla p = \boldsymbol{0} & \text{in } \Omega_f, \\ \operatorname{div} \boldsymbol{u} = 0 & \text{in } \Omega_f, \\ -\nabla \cdot \sigma_s(\boldsymbol{d}) = \boldsymbol{0} & \text{in } \Omega_s, \\ \boldsymbol{u} = \boldsymbol{0} & \text{on } \Sigma, \\ \sigma_s(\boldsymbol{d})\boldsymbol{n} = \nu\nabla \boldsymbol{u}\,\boldsymbol{n} - p\boldsymbol{n} & \text{on } \Sigma, \end{cases} \tag{1}$$

where $\Sigma = \partial\Omega_f \cap \partial\Omega_s$ and $\boldsymbol{n}$ is the unit normal on Σ outgoing from Ω_f and incoming into Ω_s, exactly as specified in the brief.
For the discussion of the implementation, it is convenient to introduce the boundary partition

$$\partial\Omega_f = \Gamma_{\text{in}} \cup \Gamma_{\text{out}} \cup \Gamma_w^f \cup \Sigma, \qquad \partial\Omega_s = \Gamma_D^s \cup \Gamma_N^s \cup \Sigma,$$

with $\Gamma_w^f = \Gamma^f_{\text{LeftWallFluid}} \cup \Gamma^f_{\text{RightWallFluid}}$, $\Gamma_D^s = \Gamma_{\text{BottomWall}}$, and $\Gamma_N^s = \Gamma_{\text{LeftWallSolid}} \cup \Gamma_{\text{RightWallSolid}}$.
The unknowns are the fluid velocity $\boldsymbol{u}$, the fluid pressure p, and the solid displacement $\boldsymbol{d}$. In the code, the stationary Stokes equations are solved in the fluid subdomain:

$$\begin{cases} -\nu\Delta \boldsymbol{u} + \nabla p = \boldsymbol{0} & \text{in } \Omega_f, \\ \operatorname{div} \boldsymbol{u} = 0 & \text{in } \Omega_f. \end{cases} \tag{2}$$

The viscosity is set in the code to $\nu = 1$.
The solid is modeled by the linear elasticity operator associated with the stress tensor

$$\sigma_s(\boldsymbol{d}) = 2\mu\,\varepsilon(\boldsymbol{d}) + \lambda\,\operatorname{div}(\boldsymbol{d})\,I, \qquad \varepsilon(\boldsymbol{d}) = \frac{\nabla \boldsymbol{d} + \nabla \boldsymbol{d}^T}{2},$$

so that the strong form reads

$$-\nabla \cdot \sigma_s(\boldsymbol{d}) = \boldsymbol{0} \qquad \text{in } \Omega_s, \tag{3}$$

with Lamé-type parameters $\mu = 1$ and $\lambda = 1$.
The imposed boundary data are:

$$\boldsymbol{u} = \boldsymbol{u}_{\text{in}} \qquad \text{on } \Gamma_{\text{in}}, \tag{4}$$

$$(\nu\nabla \boldsymbol{u} - pI)\,\boldsymbol{n} = \boldsymbol{0} \qquad \text{on } \Gamma_{\text{out}}, \tag{5}$$

$$\boldsymbol{d} = \boldsymbol{0} \qquad \text{on } \Gamma_D^s. \tag{6}$$

The inflow profile implemented in the class `InletVelocity` is parabolic and points downward:

$$\boldsymbol{u}_{\text{in}}(x) = \begin{bmatrix} 0 \\ -4U_{\max}\dfrac{x}{L}\left(1 - \dfrac{x}{L}\right) \end{bmatrix}, \qquad U_{\max} = 1, \quad L = 0.5. \tag{7}$$

At the interface, the implemented coupling reads

$$\boldsymbol{u} = \boldsymbol{0} \qquad \text{on } \Sigma, \tag{8}$$

$$\sigma_s(\boldsymbol{d})\boldsymbol{n} = \nu\nabla \boldsymbol{u}\,\boldsymbol{n} - p\,\boldsymbol{n} \qquad \text{on } \Sigma, \tag{9}$$

which matches the form used in the project brief. The report distinguishes Equation (1), which summarizes the assigned problem, from Equation (2)–Equation (9), which make explicit the constitutive law and the operator assembled by the present implementation.

4.2. Finite element spaces

The solver uses simplicial finite elements and an `hp::FECollection` with two active finite element systems:

$$\mathcal{F}_f = \left[(P_2)^2, P_1, 0\right] \qquad \text{on fluid cells,}$$

$$\mathcal{F}_s = \left[0, 0, (P_2)^2\right] \qquad \text{on solid cells.}$$

The inactive components are represented by `FE_Nothing`. Therefore, the global unknown still has five components,

$$(u_x, u_y, p, d_x, d_y),$$

but each cell only activates the subset associated with the corresponding material.
This choice matches the values passed by the executable:

$$k_u = 2, \qquad k_p = 1, \qquad k_d = 2.$$

Hence the fluid uses a Taylor-Hood pair, while the solid displacement is approximated with piecewise quadratic continuous elements. The choice of the mixed fluid space follows the standard stable Taylor-Hood construction for incompressible flow problems; see, for example, [2].

4.3. Weak form and algebraic structure

Using test functions $\boldsymbol{v}$ for the fluid velocity, q for the pressure, and $\boldsymbol{w}$ for the solid displacement, the assembled contributions are:

- the Stokes bilinear form on Ω_f,

$$a_f((\boldsymbol{u}, p), (\boldsymbol{v}, q)) = \int_{\Omega_f} \nu \nabla \boldsymbol{u} : \nabla \boldsymbol{v} - \int_{\Omega_f} p \operatorname{div} \boldsymbol{v} - \int_{\Omega_f} q \operatorname{div} \boldsymbol{u};$$

- the solid bilinear form on Ω_s,

$$a_s(\boldsymbol{d}, \boldsymbol{w}) = \int_{\Omega_s} 2\mu\, \varepsilon(\boldsymbol{d}) : \varepsilon(\boldsymbol{w}) + \int_{\Omega_s} \lambda \operatorname{div} \boldsymbol{d} \operatorname{div} \boldsymbol{w};$$

- the interface term that transfers the fluid traction to the solid,

$$c((\boldsymbol{u}, p), \boldsymbol{w}) = \int_{\Sigma} (\nu \nabla \boldsymbol{u}\, \boldsymbol{n} - p\, \boldsymbol{n}) \cdot \boldsymbol{w}.$$

After discretization, the assembled linear system has the block form

$$\begin{bmatrix} A & B^T & 0 \\ B & 0 & 0 \\ C_u & C_p & K \end{bmatrix} \begin{bmatrix} U \\ P \\ D \end{bmatrix} = \begin{bmatrix} F_u \\ 0 \\ 0 \end{bmatrix}. \tag{10}$$

Here A is the velocity stiffness matrix, B encodes the incompressibility constraint, K is the solid stiffness matrix, and C_u, C_p represent the interface traction coupling from the fluid to the solid. The right-hand side is generated only through boundary-condition elimination because the implementation has no volumetric forcing term.
The lower-triangular structure in Equation (10) is the correct algebraic counterpart of the assigned fixed-domain model: the fluid block is solved independently of the solid displacement, while the solid block depends on the fluid traction. In other words, the discrete system is monolithic in storage and assembly, but its coupling pattern reflects the small-displacement assumptions of Project 9. Since the normal in the project statement points from the fluid into the solid, whereas the solid weak form uses the outward solid normal, the implementation stores the fluid-to-solid traction in the coupling blocks and then solves

$$KD = -C_u U - C_p P.$$

This sign convention gives the solid right-hand side associated with the outward normal of Ω_s.

4.4. Stability and accuracy remarks

From the viewpoint of the discrete ingredients, the fluid block relies on the classical Taylor-Hood pair, which is a standard inf-sup stable choice for Stokes-type problems on simplicial meshes [2]. The solid block is associated with a coercive bilinear form once the displacement is fixed on the bottom boundary, since the nonempty Dirichlet portion removes rigid-body modes and yields control of the H^1 norm. The pressure nullspace of the Stokes problem is removed in the implementation by pinning one deterministic reference pressure degree of freedom selected from a fixed fluid cell.
As far as validation is concerned, the evidence is stronger than a single qualitative plot but still narrower than a full convergence study. The solver has been checked for MPI consistency on 1, 2, 3, and 4 ranks, and the 1-rank versus 2-rank comparison yields a relative L^2 discrepancy of approximately 7.39×10^{-6} over the sampled solution fields, see Table 3. This supports correctness of the distributed implementation.

Moreover, the simulations used for the performance analysis were carried out on a globally refined mesh, obtained by applying `refine_global(2)`. This refinement increases the number of elements and degrees of freedom, making the test problem more representative from a computational point of view and more suitable for a meaningful strong scalability study, see Section 7.

5. Implementation in deal.II

5.1. Program structure

The executable `exerciseFSI.cpp` defines the mesh path and the polynomial degrees, constructs an `FSIProblem` object, measures the execution time of each stage through MPI time and barrier functions, and then calls the four main stages of the simulation:

1. `setup()`,
2. `assemble()`,
3. `solve()`,
4. `output()`.

This keeps the entry point small and concentrates the numerical logic inside a single class.

5.2. Setup phase

The setup stage performs the following operations:

- reads the Gmsh mesh with `GridIn` into a `parallel::shared::Triangulation`;
- checks that the mesh file exists and contains nonzero numbers of nodes and elements;
- applies a global mesh refinement through `refine_global(2)`, increasing the number of elements and degrees of freedom;
- creates the fluid and solid finite element systems and stores them in an `hp::FECollection`;
- assigns `active_fe_index = 0` to fluid cells and `active_fe_index = 1` to solid cells based on the material id;
- distributes degrees of freedom and renumbers them component-wise into velocity, pressure, and displacement blocks;
- builds locally relevant constraints, including hanging nodes, Dirichlet data, interface velocity constraints, and a deterministic pressure pinning condition;
- initializes the distributed block sparsity pattern, the Trilinos block matrix, the pressure mass matrix used in the preconditioner, and the distributed block vectors.

The mesh checks make failures easier to diagnose. If the mesh is missing or empty, the code produces a targeted error message instead of failing later inside `deal.II` with a generic I/O exception.

5.3. Assembly phase

The assembly uses `hp::FEValues` and `hp::FEFaceValues` so that fluid and solid cells can be processed in a single loop even though they activate different finite element systems. All element loops are restricted to locally owned cells, and the distributed matrices and vectors are finalized through Trilinos compression operations after assembly.

For fluid cells, the code assembles the stationary Stokes terms:

- diffusion of the velocity,
- pressure contribution to the momentum equation,
- divergence constraint.

No explicit Neumann term is added at the outflow, so that boundary remains a natural boundary of the weak form.

For solid cells, the code assembles the linear elastic contribution based on the symmetric-gradient form described in Equation (3). On faces lying on the interface with a fluid neighbor, the code evaluates the fluid traction using the fluid gradient and pressure and adds the corresponding interface contribution to the solid rows of the global matrix. The interface quadrature points are matched in physical coordinates before evaluating traces from the two neighboring cells, so the assembly does not rely on both cells storing the common face with the same local orientation. In parallel, the pressure mass matrix is assembled on the fluid cells to provide a Schur-complement approximation for the Stokes preconditioner.

Dirichlet conditions are collected in `AffineConstraints` and enforced during local-to-global distribution. In particular:

- the inflow profile of Equation (7) is enforced on `Inflow`;
- zero solid displacement is imposed on `BottomWall`;
- the solid and fluid lateral walls are left natural in this setup.

5.4. Linear solver and output

The solver uses distributed Trilinos containers and exploits the lower-triangular structure of Equation (10) explicitly:

1. the Stokes block
$$\begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix}$$
is solved with `FGMRES`;
2. the corresponding preconditioner is Schur-type: AMG is used on the velocity block, while the pressure Schur complement is approximated by the pressure mass matrix and preconditioned with Jacobi;
3. after the fluid unknowns are computed, the solid block
$$KD = -C_u U - C_p P$$
is solved with `SolverCG` and AMG.

This choice is consistent with the fixed-domain FSI structure of Project 9 and avoids applying a generic monolithic solver to an indefinite saddle-point system. The Stokes iterations and solid iterations are recorded at runtime for diagnostics, see Table 2.

The post-processing stage exports a parallel `PVTU`/`VTU` data set rooted at `output-FSIProblem_0000.pvtu`. The output contains the fields:

- `velocity`,
- `pressure`,
- `displacement`,
- `partitioning`.

In addition, each run writes a report file `output-FSIProblem_np#.report` with global norms and extrema, together with a rank-independent sampled solution snapshot `output-FSIProblem_np#.solution.csv`. The companion script `scripts/compare_solution_runs.sh` compares two snapshots and reports weighted L^2 and L^∞ discrepancies between runs, see Table 3.

6. Numerical Results and Parallel Verification

A complete run confirms that the workflow executes successfully from mesh import to distributed output. In the tested configuration, the program reports 45024 active cells and 194629 total degrees of freedom. The mesh is generated by Gmsh from the geometry file in `mesh/`, while `deal.II` reports only the active two-dimensional cells used in the computation.

6.1. MPI-rank consistency

The most important validation for the distributed code is rank independence. Since the solver uses MPI-distributed data structures and block preconditioners, we verify that changing the number of MPI ranks does not change the physical solution beyond solver tolerance.

Table 2 summarizes the runs performed on 1, 2, 3, and 4 MPI ranks. The reference pressure degree of freedom changes its global number because the DoF numbering depends on the partition, but it is selected from the same reference fluid cell in all cases; the cell center printed by the code is $(0.00165058, 0.304066)$.

The Stokes and solid iteration counts remain essentially stable across the tested rank counts, with only one-iteration variations. These small differences are expected in parallel Krylov solvers, since floating-point reductions, matrix-vector products, and AMG preconditioners may depend slightly on the MPI partition. The reported L^2 norms of velocity, pressure, and displacement remain unchanged at the displayed precision, indicating that the physical solution is rank-independent up to solver tolerance. Therefore, changing from 1 MPI rank to 2, 3, or 4 produces solutions whose differences are compatible with the accuracy requested from the iterative solvers.

Table 2: MPI consistency summary for the distributed solver.

MPI ranks	Pressure ref. DoF	Stokes iter.	Solid iter.	$\|\boldsymbol{u}\|_{L^2}$	$\|p\|_{L^2}$	$\|\boldsymbol{d}\|_{L^2}$
1	110140	28	218	0.283578	3.44171	0.768444
2	101880	27	218	0.283578	3.44171	0.768444
3	110539	28	218	0.283578	3.44171	0.768444
4	107116	27	219	0.283578	3.44171	0.768444

To check the distributed correctness more quantitatively, the script `compare_solution_runs.sh` was applied to the sampled solution snapshots generated by the 1-rank and 2-rank runs. The resulting discrepancies are reported in Table 3. All relative L^2 errors are of order 10^{-6}, which is consistent with solver tolerance rather than a physical difference between the runs.

Table 3: Weighted discrepancies between the 1-rank and 2-rank sampled solutions.

Field	L^2 error	L^∞ error	Relative L^2 error
Velocity	4.28×10^{-7}	3.16×10^{-6}	1.51×10^{-6}
Pressure	2.57×10^{-5}	3.41×10^{-3}	7.45×10^{-6}
Displacement	4.86×10^{-6}	2.44×10^{-5}	6.34×10^{-6}
Total	2.61×10^{-5}	3.41×10^{-3}	7.39×10^{-6}

The present report therefore supports the statement that the MPI implementation is correct in the sense of rank-independent numerical results. The parallel performance of the code is instead assessed separately in the strong-scalability study reported in Section 7.

6.2. Velocity field

The velocity magnitude and the associated glyph visualization are shown in Figure 3. The largest velocity values appear near the inflow, consistently with the imposed parabolic inlet profile. The solid region blocks part of the motion and redirects the flow around the interface. Since the fluid velocity is constrained to zero on the fluid-solid interface, the flow is strongly damped near this boundary. The glyph representation, on the right, highlights the formation of recirculation regions close to the interface. No rank-dependent discontinuities were observed in the parallel ParaView output.

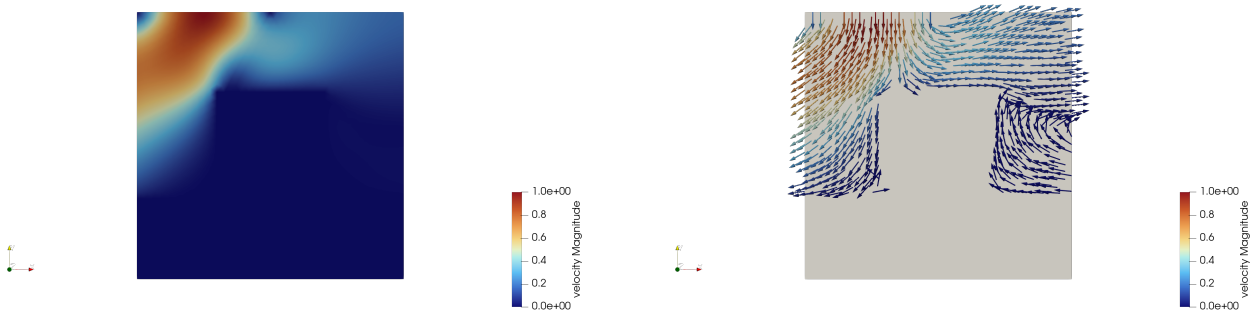


Figure 3: Velocity field: magnitude plot (left) and glyph representation (right).

6.3. Pressure field

The pressure field is shown in Figure 4. A smooth pressure variation is observed across the fluid region, with stronger gradients close to the interface and near the inflow section. Away from the interface, the pressure field becomes more uniform. This is coherent with the Stokes character of the model and with the role of pressure in enforcing incompressibility and balancing the redirection of the flow around the solid. Since the discrete pressure is defined up to an additive constant, the implementation fixes a deterministic reference pressure degree of freedom; the plotted field should therefore be interpreted relative to that normalization.

Figure 4: Pressure distribution in the fluid region.

6.4. Displacement field

The displacement magnitude and glyph representation are shown in Figure 5. The displacement is confined to the solid subdomain, as expected from the `FE_Nothing` construction. The largest deformation is concentrated near the interface, where the fluid traction acts on the solid. The clamped bottom wall suppresses motion at the support, while the solid lateral walls remain traction-free in this setup. The displacement norms reported in Table 2 remain stable across all tested MPI counts.

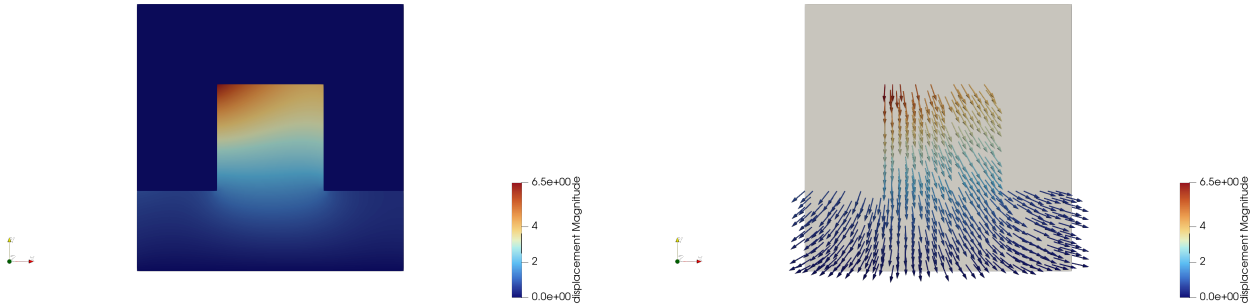


Figure 5: Solid displacement: magnitude plot (left) and glyph representation (right).

Overall, the numerical results are consistent with the implemented model: a fixed-domain Stokes solve produces interface tractions that generate a nontrivial deformation of the solid, while the fluid itself is not updated by the resulting displacement, exactly as prescribed by the small-displacement project formulation. The combination of visual inspection and MPI-consistency checks provides a quantitative validation of the distributed code path, while the parallel performance of the implementation is assessed separately through the strong-scalability study in the following section.

7. Strong Scalability Analysis

A strong scalability study was performed in order to evaluate the parallel performance of the implementation. The computational problem was kept fixed, while the number of MPI processes was varied. For each run, the wall-clock time was measured separately for the main phases of the code, with particular attention to the solve phase and to the total execution time.
The speedup is defined as

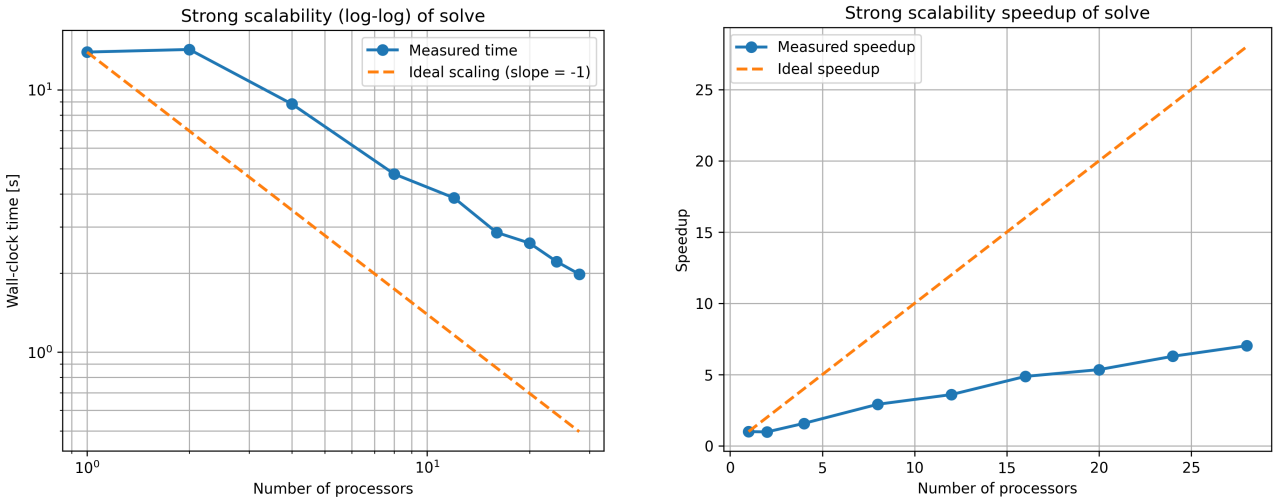
$$S_p = \frac{T_1}{T_p},$$

where T_1 is the execution time obtained with one MPI process and T_p is the corresponding time obtained with p MPI processes. The ideal strong scaling behavior is represented by $S_p = p$, or equivalently by a line with slope -1 in the log–log plot of execution time versus number of processors.
Preliminary tests performed on the coarser mesh showed limited scalability due to the relatively small problem size. Therefore, the timing study was conducted on the refined mesh, which provides a larger computational workload and allows the benefits of parallel execution to emerge more clearly.

The tests were run by using the student access to the HPC infrastructure of the department of Mathematics, using the general compute nodes, divided into 5 CPU nodes (112 hardware threads, 0.5 TB RAM, 8.7 TB local scratch) and 2 multi-GPU nodes (112 hardware threads, 1 TB RAM, 10.5 TB local scratch, and 2 NVIDIA L4 GPUs each).

7.1. Solve phase

Figure 6 reports the strong scalability results for the solve phase. The measured speedup increases with the number of processors, showing that the iterative block solver benefits from the parallel distribution of the algebraic system. In particular, the use of `Trilinos` parallel matrices and vectors allows the matrix-vector products and the preconditioned Krylov iterations to be distributed among MPI ranks.
However, the measured speedup remains below the ideal curve. This deviation is mainly due to communication overhead, synchronization costs, and the non-perfect scalability of the preconditioners.



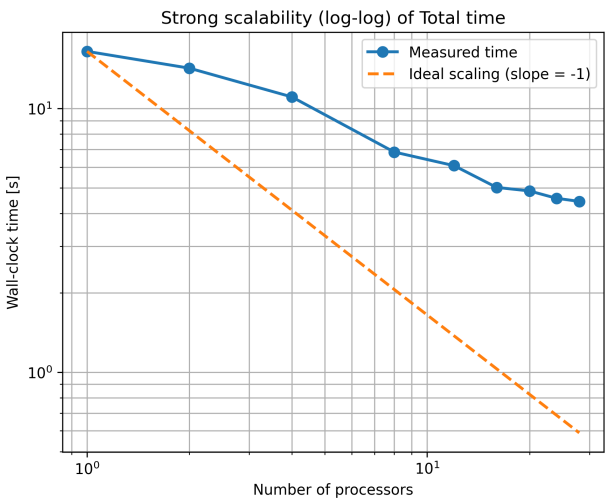
(a) Log–log wall-clock time.

(b) Speedup.

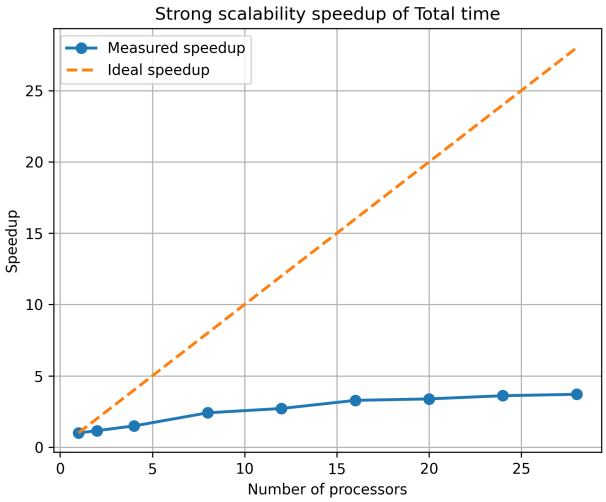
Figure 6: Strong scalability of the solve phase.

7.2. Total execution time

The scalability of the total execution time is shown in Figure 7. The total time also decreases when increasing the number of processors, but the speedup saturates earlier than in the solve phase. This behavior is due to the fact that the total runtime includes not only the linear solver, but also setup, assembly and output-related operations.
In particular, the implementation relies on a `parallel::shared::Triangulation`. This choice simplifies the implementation and allows the code to run correctly in parallel, but it also implies that mesh information is replicated across MPI ranks. As the number of processors increases, the amount of local work per process decreases, while communication and synchronization overheads become increasingly relevant. Consequently, the total speedup tends to saturate for larger numbers of processors.



(a) Log–log wall-clock time.



(b) Speedup.

Figure 7: Strong scalability of the total execution time.

8. Conclusions

The project provides a complete and working pipeline for a steady fluid-structure interaction computation on a two-dimensional simplicial mesh. The main implementation choices are consistent with the implemented code: Taylor-Hood elements for the fluid, piecewise quadratic displacement for the solid, hp-based activation of material-dependent finite element systems, MPI-parallel `deal.II`/Trilinos data structures, a block solver tailored to the lower-triangular FSI operator, and parallel `PVTU` output with automatic diagnostics.
The report reflects the actual state of the software: the mesh is generated in MSH v2 format, the inlet profile is parabolic, the solid displacement degree is two, a deterministic pressure pinning strategy is used, and the distributed solver is validated through MPI-rank consistency checks on 1, 2, 3, and 4 ranks.
A strong-scalability study was also carried out on the refined mesh. The results show that the parallel implementation achieves a clear reduction in solve time as the number of MPI processes increases. The observed speedup is sublinear, as expected, due to communication costs, synchronization overheads, and the limited scalability of the preconditioners. The total execution time also decreases, although its speedup saturates earlier than that of the solve phase. This is because the total runtime also includes setup, assembly, output, and mesh-management operations, whose scalability is limited by the use of a `parallel::shared::Triangulation`.
Further improvements could be obtained by adopting a different parallel mesh strategy, aimed at reducing memory replication and improving load balance. In addition, stronger Schur-complement approximations or more advanced block preconditioners could improve the robustness and scalability of the solver for larger problems. Finally, a more general two-way ALE formulation could be considered in order to move beyond the fixed-domain small-displacement model prescribed in the project brief.

References

[1] Daniel Arndt, Wolfgang Bangerth, Bruno Blais, Todd C. Clevenger, Michael Fehling, Alexander V. Grayver, Timo Heister, Luca Heltai, Martin Kronbichler, Matthias Maier, Peter Munch, Jean-Paul Pelteret, Reza Rastak, Kristian Simon, Bruno Turcksin, and David Wells. The deal.II library, version 9.1. *Journal of Numerical Mathematics*, 27(4):203–213, 2019.

[2] Daniele Boffi, Franco Brezzi, and Michel Fortin. *Mixed Finite Element Methods and Applications*, volume 44 of *Springer Series in Computational Mathematics*. Springer, Heidelberg, 2013.

[3] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009.